Tutorial 3
Bubble Sort, Selection Sort & Insertion Sort

Overview

- This tutorial explains Bubble Sort, Selection Sort, and Insertion Sort in C++ with tracing code to visualize each step. It demonstrates how each algorithm sorts an array by repeatedly comparing, swapping, or shifting elements into their correct positions.

Part 1: Bubble Sort

1. Open VS Code. Then add 'New file' and name the new file as 'bubbleSort.cpp'.
2. Then write down the following codes:

```cpp
#include <iostream>
#include <string>
using namespace std;

// Bubble sort for integer arrays
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        cout << "Integer Array Pass " << i + 1 << ":\n";
        for (int j = 0; j < n - i - 1; j++) {
            cout << "  Comparing " << arr[j] << " and " << arr[j + 1] << ": ";
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                cout << "Swapped => ";
            } else {
                cout << "No Swap => ";
            }
            // Print the current state of the array
            for (int k = 0; k < n; k++) cout << arr[k] << " ";
            cout << endl;
        }
        cout << "End of Integer Array Pass " << i + 1 << ":\n";
        for (int k = 0; k < n; k++) cout << arr[k] << " ";
        cout << "\n\n";
    }
}
```

```
// Bubble sort for string arrays
void bubbleSort(string arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        cout << "String Array Pass " << i + 1 << ":\n";
        for (int j = 0; j < n - i - 1; j++) {
            cout << "  Comparing \"" << arr[j] << "\" and \"" << arr[j + 1] << "\": ";
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                string temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                cout << "Swapped => ";
            } else {
                cout << "No Swap => ";
            }
            // Print the current state of the array
            for (int k = 0; k < n; k++) cout << "\"" << arr[k] << "\" ";
            cout << endl;
        }
        cout << "End of String Array Pass " << i + 1 << ":\n";
        for (int k = 0; k < n; k++) cout << "\"" << arr[k] << "\" ";
        cout << "\n\n";
    }
}

int main() {
    int intArr[] = {64, 25, 12, 22, 11};
    int intSize = sizeof(intArr) / sizeof(intArr[0]);
    cout << "Sorting Integer Array:\n";
    bubbleSort(intArr, intSize);

    string strArr[] = {"apple", "orange", "banana", "grape", "cherry"};
    int strSize = sizeof(strArr) / sizeof(strArr[0]);
    cout << "Sorting String Array:\n";
    bubbleSort(strArr, strSize);

    return 0;
}
```

3. **Bubble Sort** is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted, gradually "bubbling" the largest unsorted element to its correct position in each pass.

Compile and Run

4. Press F5 to run the code.
5. The output will show the:

```
Sorting Integer Array:
Integer Array Pass 1:
  Comparing 64 and 25: Swapped => 25 64 12 22 11
  Comparing 64 and 12: Swapped => 25 12 64 22 11
  Comparing 64 and 22: Swapped => 25 12 22 64 11
  Comparing 64 and 11: Swapped => 25 12 22 11 64
End of Integer Array Pass 1:
25 12 22 11 64

Integer Array Pass 2:
  Comparing 25 and 12: Swapped => 12 25 22 11 64
  Comparing 25 and 22: Swapped => 12 22 25 11 64
  Comparing 25 and 11: Swapped => 12 22 11 25 64
End of Integer Array Pass 2:
12 22 11 25 64

Integer Array Pass 3:
  Comparing 12 and 22: No Swap => 12 22 11 25 64
  Comparing 22 and 11: Swapped => 12 11 22 25 64
End of Integer Array Pass 3:
12 11 22 25 64

Integer Array Pass 4:
  Comparing 12 and 11: Swapped => 11 12 22 25 64
End of Integer Array Pass 4:
11 12 22 25 64

Sorting String Array:
String Array Pass 1:
  Comparing "apple" and "orange": No Swap => "apple" "orange" "banana" "grape" "cherry"
  Comparing "orange" and "banana": Swapped => "apple" "banana" "orange" "grape" "cherry"
  Comparing "orange" and "grape": Swapped => "apple" "banana" "grape" "orange" "cherry"
  Comparing "orange" and "cherry": Swapped => "apple" "banana" "grape" "cherry" "orange"
End of String Array Pass 1:
"apple" "banana" "grape" "cherry" "orange"

String Array Pass 2:
  Comparing "apple" and "banana": No Swap => "apple" "banana" "grape" "cherry" "orange"
  Comparing "banana" and "grape": No Swap => "apple" "banana" "grape" "cherry" "orange"
  Comparing "grape" and "cherry": Swapped => "apple" "banana" "cherry" "grape" "orange"
End of String Array Pass 2:
"apple" "banana" "cherry" "grape" "orange"

String Array Pass 3:
  Comparing "apple" and "banana": No Swap => "apple" "banana" "cherry" "grape" "orange"
  Comparing "banana" and "cherry": No Swap => "apple" "banana" "cherry" "grape" "orange"
End of String Array Pass 3:
"apple" "banana" "cherry" "grape" "orange"

String Array Pass 4:
  Comparing "apple" and "banana": No Swap => "apple" "banana" "cherry" "grape" "orange"
End of String Array Pass 4:
"apple" "banana" "cherry" "grape" "orange"
```

6. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.

Part 2: Selection Sort

1. Create ‘New file’ and name it as ‘selectionSort.cpp’.
2. Then write down the following codes:

```cpp
#include <iostream>
#include <string>
using namespace std;

// Selection Sort for integer marks array (finding the highest index)
void selectionSort(int marks[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i; // Initialize the index of the maximum element
        cout << "Marks Array Pass " << i + 1 << ":\n";
        for (int j = i + 1; j < n; j++) {
            cout << "  Comparing " << marks[j] << " with " << marks[maxIdx];
            if (marks[j] > marks[maxIdx]) { // Change to find the maximum
                maxIdx = j; // Update the index of the maximum element
                cout << " => New max at index " << maxIdx;
            }
            cout << endl;
        }
        // Swap the found maximum element with the first element
        if (maxIdx != i) {
            int temp = marks[maxIdx];
            marks[maxIdx] = marks[i];
            marks[i] = temp;
        }
        // Display the marks array state after each pass
        for (int k = 0; k < n; k++) cout << marks[k] << " ";
        cout << "\n\n";
    }
}
```

```
// Selection Sort for string names array (finding the highest index)
void selectionSort(string names[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i; // Initialize the index of the maximum element
        cout << "Names Array Pass " << i + 1 << ":\n";
        for (int j = i + 1; j < n; j++) {
            cout << "  Comparing \"" << names[j] << "\" with \"" << names[maxIdx] << "\"";
            if (names[j] > names[maxIdx]) { // Change to find the maximum
                maxIdx = j; // Update the index of the maximum element
                cout << " => New max at index " << maxIdx;
            }
            cout << endl;
        }
        // Swap the found maximum element with the first element
        if (maxIdx != i) {
            string temp = names[maxIdx];
            names[maxIdx] = names[i];
            names[i] = temp;
        }
        // Display the names array state after each pass
        for (int k = 0; k < n; k++) cout << "\"" << names[k] << "\" ";
        cout << "\n\n";
    }
}

int main() {
    int marks[] = {85, 74, 92, 67, 89};
    int marksSize = sizeof(marks) / sizeof(marks[0]);
    cout << "Sorting marks array:\n";
    selectionSort(marks, marksSize);

    string names[] = {"Li Wei", "Amit", "Hiroshi", "Nguyen", "Siti"};
    int namesSize = sizeof(names) / sizeof(names[0]);
    cout << "Sorting names array:\n";
    selectionSort(names, namesSize);

    return 0;
}
```

3. **Selection Sort** is an algorithm that divides the list into a sorted and an unsorted region. It repeatedly selects the smallest (or largest) element from the unsorted portion and swaps it with the first unsorted element, gradually expanding the sorted region.

Compile and Run

4. Press F5 to run the code.
5. The output will show the:

```
Sorting marks array:
Marks Array Pass 1:
  Comparing 74 with 85
  Comparing 92 with 85 => New max at index 2
  Comparing 67 with 92
  Comparing 89 with 92
92 74 85 67 89

Marks Array Pass 2:
  Comparing 85 with 74 => New max at index 2
  Comparing 67 with 85
  Comparing 89 with 85 => New max at index 4
92 89 85 67 74

Marks Array Pass 3:
  Comparing 67 with 85
  Comparing 74 with 85
92 89 85 67 74

Marks Array Pass 4:
  Comparing 74 with 67 => New max at index 4
92 89 85 74 67

Sorting names array:
Names Array Pass 1:
  Comparing "Amit" with "Li Wei"
  Comparing "Hiroshi" with "Li Wei"
  Comparing "Nguyen" with "Li Wei" => New max at index 3
  Comparing "Siti" with "Nguyen" => New max at index 4
"Siti" "Amit" "Hiroshi" "Nguyen" "Li Wei"

Names Array Pass 2:
  Comparing "Hiroshi" with "Amit" => New max at index 2
  Comparing "Nguyen" with "Hiroshi" => New max at index 3
  Comparing "Li Wei" with "Nguyen"
"Siti" "Nguyen" "Hiroshi" "Amit" "Li Wei"

Names Array Pass 3:
  Comparing "Amit" with "Hiroshi"
  Comparing "Li Wei" with "Hiroshi" => New max at index 4
"Siti" "Nguyen" "Li Wei" "Amit" "Hiroshi"

Names Array Pass 4:
  Comparing "Hiroshi" with "Amit" => New max at index 4
"Siti" "Nguyen" "Li Wei" "Hiroshi" "Amit"
```

6. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.

Part 3: Insertion Sort

1. Create ‘New file’ and name it as ‘insertionSort.cpp’.
2. Then write down the following codes:

```cpp
#include <iostream>
#include <string>
using namespace std;

// Insertion Sort for integer temperatures array
void insertionSort(int temperatures[], int n) {
    for (int i = 1; i < n; i++) {
        int key = temperatures[i];
        int j = i - 1;

        // Move elements greater than key to one position ahead
        while (j >= 0 && temperatures[j] > key) {
            temperatures[j + 1] = temperatures[j];
            j--;
        }
        temperatures[j + 1] = key;

        // Display the temperatures array state after each insertion
        cout << "Pass " << i << ": Insert " << key << " into the sorted section:\n";
        for (int k = 0; k < n; k++) {
            if (k <= i) {
                cout << "[" << temperatures[k] << "] "; // Highlight sorted part
            } else {
                cout << temperatures[k] << " "; // Unsorted part
            }
        }
        cout << "\n\n";
    }
```

```
// Insertion Sort for string colors array
void insertionSort(string colors[], int n) {
    for (int i = 1; i < n; i++) {
        string key = colors[i];
        int j = i - 1;

        // Move elements greater than key to one position ahead
        while (j >= 0 && colors[j] > key) {
            colors[j + 1] = colors[j];
            j--;
        }
        colors[j + 1] = key;

        // Display the colors array state after each insertion
        cout << "Pass " << i << ": Insert \"" << key << "\" into the sorted section:\n";
        for (int k = 0; k < n; k++) {
            if (k <= i) {
                cout << "[\"" << colors[k] << "\"] "; // Highlight sorted part
            } else {
                cout << "\"" << colors[k] << "\" "; // Unsorted part
            }
        }
        cout << "\n\n";
    }
}

int main() {
    int temperatures[] = {30, 22, 25, 35, 28};
    int tempSize = sizeof(temperatures) / sizeof(temperatures[0]);
    cout << "Sorting temperatures array:\n";
    insertionSort(temperatures, tempSize);

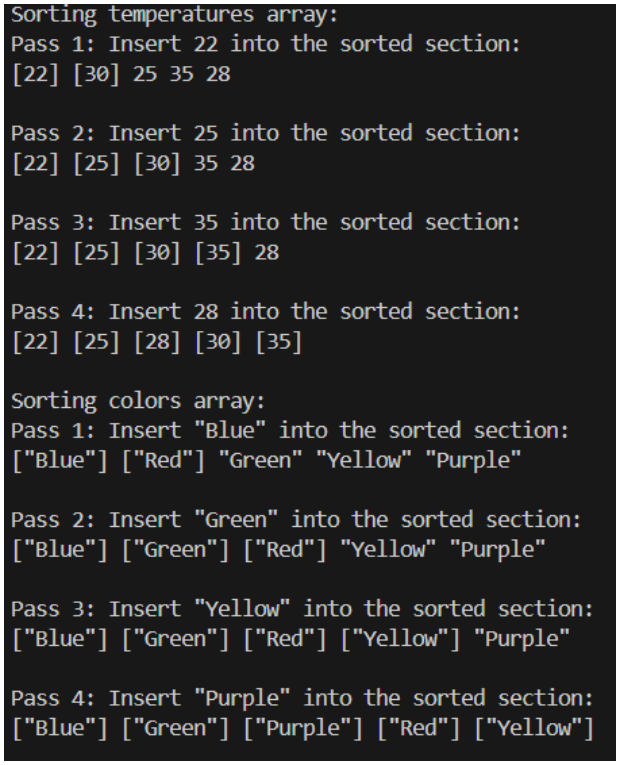
    string colors[] = {"Red", "Blue", "Green", "Yellow", "Purple"};
    int colorsSize = sizeof(colors) / sizeof(colors[0]);
    cout << "Sorting colors array:\n";
    insertionSort(colors, colorsSize);

    return 0;
}
```

3. **Insertion Sort:** is an algorithm that builds a sorted array one element at a time by taking an unsorted element and placing it in its correct position within the sorted portion, gradually expanding the sorted region.

Compile and Run

4. Press F5 to run the code.
5. The output will show the:



6. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.